Lab 1: RV64 内核引导

1 实验目的

学习RISC-V相关知识，了解OpenSBI平台，实现sbi调用函数，封装打印函数，并利用Makefile来完成对整个工程的管理。

为降低实验难度，我们选择使用OpenSBI作为BIOS来进行机器启动时m模式下的硬件初始化与寄存器设置，并使用OpenSBI所提供的接口完成诸如字符打印等的操作。

2 实验内容及要求

- 阅读RISC-V中文手册，学习RISC-V相关知识
- 学习Makefile编写规则，补充Makefile文件使得项目成功运行
- 了解OpenSBI的运行原理，编写代码通过sbi调用实现字符串的打印

请各位同学独立完成实验，任何抄袭行为都将使本次实验判为0分。请查看文档尾部附录部分的背景知识介绍，跟随实验步骤完成实验，以截图的方式记录命令行的输入与输出，注意进行适当的代码注释。如有需要，请对每一步的命令以及结果进行必要的解释。

在最终提交的实验报告中，**请自行删除附录部分内容**，并命名为"**学号_姓名_lab1.pdf**"，代码文件请根据给出的结构整理好后打包命名为"**学号_姓名_lab1**"，分开上传至学在浙大平台。

3 实验步骤

3.1 搭建实验环境（10%）

按照以下方法创建新的容器，并建立volume映射(参考资料)，在本地编写代码，并在容器内进行编译检查。未来的实验中同样需要用该方法搭建相应的实验环境，但不再作具体指导，请理解每一步的命令并自行更新相关内容。**注意及时关闭运行中的容器。**

1.创建容器并建立映射关系

```
### 首先新建自己本地的工作目录(比如lab1)并进入
$ mkdir lab1
$ cd lab1
$ pwd
~/.../lab1

### 查看docker已有镜像(与lab0同一个image)
$ docker image ls
REPOSITORY          TAG                 IMAGE ID            CREATED
SIZE
oslab               2020                678605140682        46 hours ago
 2.89GB

### 创建新的容器，同时建立volume映射
$ docker run -it -v `pwd`:/home/oslab/lab1 -u oslab -w /home/oslab 6786
/bin/bash
oslab@3c1da3906541:~$
```

【请将该步骤中命令的输入及运行结果截图附在此处，接下来每一步同理。】

前面的输入都一样

```
PS C:\Users\12638\Desktop\operating system\lab> mkdir lab1


    目录: C:\Users\12638\Desktop\operating system\lab


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
d-----         2021/10/1     10:15                lab1


PS C:\Users\12638\Desktop\operating system\lab> cd lab1
PS C:\Users\12638\Desktop\operating system\lab\lab1> pwd

Path
----
C:\Users\12638\Desktop\operating system\lab\lab1


PS C:\Users\12638\Desktop\operating system\lab\lab1> docker image ls
REPOSITORY                                          TAG                           IMAGE ID       CREATED
IZE
ljy/oslab                                           v1                            fa20c284c064   11 days ago
.89GB
tensorflow1.13.1                                    1.0                           8906e510de1a   6 weeks ago
.96GB
1263810658/pytorch1.8.1withgit.cuda10.2.cudnn7.runtime   latest                   5da714cfe33c   2 months ago
.57GB
pytorch/pytorch                                     1.8.1-cuda10.2-cudnn7-runtime 8feef0e83aed   3 months ago
.3GB
tensorflow/tensorflow                               latest                        1d932048a281   4 months ago
.3GB
ubuntu                                              latest                        7e0aa2d69a15   5 months ago
2.7MB
mcr.microsoft.com/mssql/server                      2019-latest                   62c72d863950   6 months ago
.49GB
```

```
docker run -it -v `pwd`:/home/oslab/lab1 -u oslab -w /home/oslab fa20 /bin/bash
# 这个好像没有正确mount
docker run -it -v /c/Users/12638/Desktop/operatingSystem/lab/lab1:/home/oslab/lab1 -u oslab -w /home/oslab fa20 /bin/bash
```

就是说user name是 oslab，工作目录是/home/oslab , 把宿主机映射到容器的/home/oslab/lab1目录

```
PS C:\Users\12638\Desktop\operating system\lab\lab1>  docker run -it -v `pwd`:/home/oslab/lab1 -u oslab -w /home/oslab f
a20 /bin/bash
oslab@707d619cd1d7:~$
```

之后用exec 进入容器而不是重新run

```
docker exec -it  -u oslab -w /home/oslab musing_lovelace /bin/bash
```

有这个提示应该是mount成功了. 之前错误挂载的时候touch 会显示错误: 没有权限.

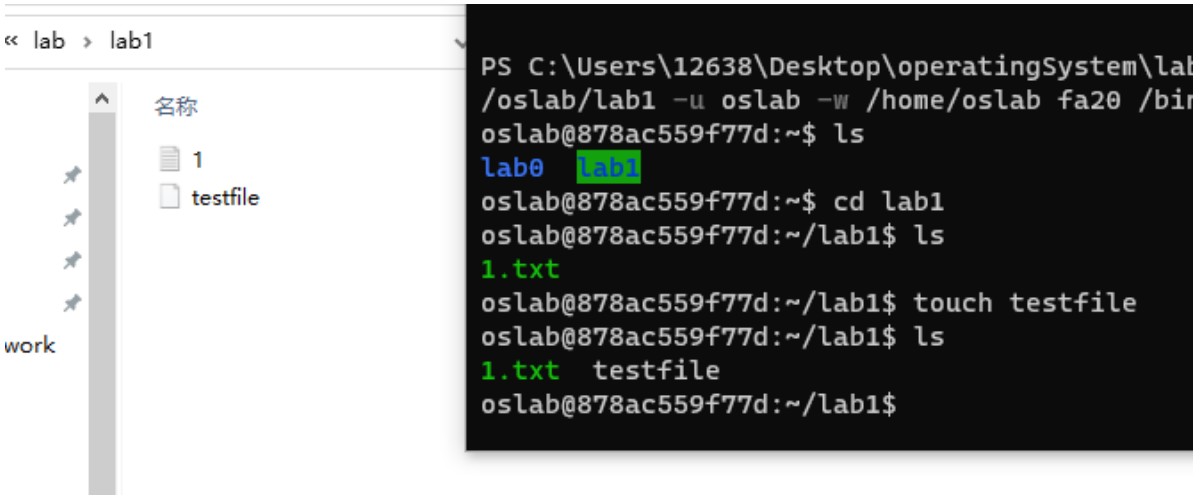
2.测试映射关系

为测试映射关系是否成功，再打开一个终端窗口，并进入`lab1`目录下

```
###Terminal2
$ touch testfile
$ ls
testfile
```

在第一个终端窗口，即容器中确认是否挂载成功。确认映射关系建立成功后，可以在本地`lab1`目录下编写实验代码，文件将映射到容器中，因此可以直接在容器中进行实验。

```
###Terminal1
### 在docker中确认是否挂载成功
oslab@3c1da3906541:~$ pwd
/home/oslab
oslab@3c1da3906541:~$ cd lab1
oslab@3c1da3906541:~$ ls
testfile
```



3.2 了解项目框架，编写MakeFile（20%）

1.编写Makefile文件

```
.
├── arch
|   └── riscv
|       ├── boot
|       ├── include
|       |   ├── print.h
|       |   ├── sbi.h
|       |   └── test.h
|       ├── kernel
|       |   ├── head.S
|       |   ├── main.c(需修改数字为学号)
|       |   ├── Makefile
|       |   ├── sbi.c（需通过内联汇编实现sbi调用）
|       |   ├── test.c
|       |   └── vmlinux.lds
|       ├── libs
|       |   ├── MAKEFILE（需编写字符串打印函数及数字打印函数）
```

```
|       |   └── print.c
|       └── Makefile
├── cp.bat
├── include
|   └── defs.h
└── Makefile
```

下载相关代码移动至`lab1`文件夹中。

项目代码结构由上图所示，通过最外层的Makefile中的"make run"命令进行编译与运行。请参考【附录A.Makefile介绍】中的基础知识，了解项目各结构中Makefile每一行命令的作用，编写`./arch/riscv/libs/Makefile`文件使得整个项目能够顺利编译，并将其填写至下述代码块中。其他步骤中的代码编写同理。

```
#lab1/arch/riscv/libs/Makefile
# YOUR MAKEFILE CODE
C_SRC       = $(sort $(wildcard *.c))
OBJ         = $(patsubst %.c,%.o,$(C_SRC))

all : $(OBJ)

%.o:%.c
    ${CC}  ${CFLAG}  -c $<

clean:
    $(shell rm *.o 2>/dev/null)
```

2.解释Makefile命令

请解释`lab1/Makefile`中的下列命令：

```
${LD} -T arch/riscv/kernel/vmlinux.lds arch/riscv/kernel/*.o arch/riscv/libs/*.o
-o vmlinux
```

含义：`${LD}`被展开为`riscv64-unknown-elf-ld`，即链接命令。`-T arch/riscv/kernel/vmlinux.lds`选项表示使用`vmlinux.lds`作为链接器脚本。`-o vmlinux`选项指定输出文件的名称为`vmlinux`，整行命令的意思是将`arch/riscv/kernel/*.o`和`arch/riscv/libs/*.o`根据`vmlinux.lds`链接形成`vmlinux`。

```
${OBJCOPY} -O binary vmlinux arch/riscv/boot/Image
```

含义：

在Linux Kernel开发中，为了对vmlinux进行精简，通常使用objcopy丢弃调试信息与符号表并生成二进制文件，. `${OBJCOPY}`被展开为`riscv64-unknown-elf-objcopy`，复制命令。`-O binary vmlinux`选项表示 把vimlinux elf文件转换为二进制bin文件.这就是Image 放在boot文件夹下

```
${MAKE} -C arch/riscv all
```

含义：
`${MAKE}`的含义可以参见这里

递归地调用make, 在arch/riscv文件夹执行make, 而且也会带有原来的参数, 比如 -t , -q , -n .

3.3 学习RISC-V相关知识及特权架构（10%）

后续实验中将持续使用RISC-V指令集相关的内容，请参考【附录B.RISC-V指令集】了解相关知识，**下载并认真阅读[RISC-V中文手册](#)**，掌握基础知识、基本命令及特权架构相关内容。

1.基础命令掌握情况

请填写命令的含义。

```
#1.加载立即数，t0=0x40000
li t0,0x40000

#2. 对于t0 中每一个为1 的位，把控制状态寄存器satp的对应位置位.
csrw satp, t0

#3.  t0 = t0 -t1
sub t0,t0,t1

#4. 64位数据传输，存储双字到x1
sd x1, 8(sp)

#5.la是risc-v指令集中的一条伪指令。伪指令，一般会被汇编器翻译成一条或者多条等价的实际指令,load address是一条地址加载指令. 把内存地址加载到sp寄存器，stack_top是一个地址标记.
la sp,stack_top
```

3.3 通过OpenSBI接口实现字符串打印函数（60%）

1.程序执行流介绍

对于本次实验，我们选择使用OpenSBI作为bios来进行机器启动时m模式下的硬件初始化与寄存器设置，并使用OpenSBI所提供的接口完成诸如字符打印等的操作。

请参考【附录B.OpenSBI介绍】了解OpenSBI平台的功能及启动方式，参考【附录D. Linux Basic】了解`vmlinux.lds`、`vmlinux`的作用，理解执行`make run`命令时程序的执行过程。

```
#make run 实际为执行lab1/Makefile中的该行命令
@qemu-system-riscv64 -nographic --machine virt -bios default -device loader,file=vmlinux,addr=0x80200000 -D log
```

QEMU模拟器完成从ZSBL到OpenSBI阶段的工作，使用-bios default选项将OpenSBI代码加载到0x80000000起始处。OpenSBI初始化完成后，跳转到0x80200000处。我们将`vmlinux`程序这一RISC-V 64-bit架构程序加载至0x80200000处运行。

`vmlinux.lds`链接脚本指定了程序的内存布局，最先加载的`.text.init`段代码为`head.s`文件的内容，顺序执行调用`main()`函数。`main()`函数调用了两个打印函数，通过`sbi_call()`向OpenSBI发起调用，完成字符的打印。

2.编写sbi_call()函数（20%）

当系统处于m模式时，对指定地址进行写操作便可实现字符的输出。但我们编写的内核运行在s模式，需要使用OpenSBI提供的接口，让运行在m模式的OpenSBI帮我们实现输出。此时，运行在s模式的内核通过`ecall`发起sbi 调用请求，RISC-V CPU 会从 s 态跳转到 m 态的 OpenSBI 固件。

执行 `ecall` 前需要指定 sbi调用的编号，传递参数。一般而言，`a7(x17)` 为 SBI 调用编号，`a0(x10)`、`a1(x11)` 和 `a2(x12)` 寄存器为sbi调用参数。**其中 `a7` 是寄存器的ABI Name，在RISC-V命令中需要使用 `x17` 进行标识，详见RISC-V中文手册中寄存器-接口名称相关的对照表。**

我们需要编写内联汇编语句以使用OpenSBI接口，给出函数定义如下：

```
typedef unsigned long long uint64_t;
uint64_t sbi_call(uint64_t sbi_type, uint64_t arg0, uint64_t arg1, uint64_t
arg2)
```

在该函数中，我们需要完成以下内容：

- 将sbi_type放到寄存器a7中，将arg0-arg2放入a0-a2中。
- 使用ecall指令发起sbi调用请求。
- OpenSBI的返回结果会放到a0中，我们需要将其取出，作为sbi_call的返回值。

请参考【附录C.内联汇编】相关知识，以内联汇编形式实现 `lab1/arch/riscv/kernel/sbi.c` 中的 `sbi_call()` 函数。

```
#lab1/arch/riscv/kernel/sbi.c
uint64_t sbi_call(uint64_t sbi_type, uint64_t arg0, uint64_t arg1, uint64_t
arg2) {
    uint64_t ret_val;
    __asm__ volatile (
        "mv x17, %[type]\n"
        "mv x10,%[src0]\n"
        "mv x11,%[src1]\n"
        "mv x12,%[src2]\n"
        "ecall\n"
        "mv %[dest],x10"
        : [dest] "=r" (ret_val)
        : [type] "r" (sbi_type), [src0] "r" (arg0), [src1] "r" (arg1), [src2]
"r" (arg2)
        :
   );
    return ret_val;
}
```

向 `sbi_call()` 传入不同的 `sbi_type` 调用不同功能的OpenSBI接口，其对应关系如下表所示，其中每个函数的具体使用方法可参考[OpenSBI 文档]。 为了利用OpenSBI接口打印字符，我们需要向 `sbi_call()` 函数传入 `sbi_type=1` 以调用 `sbi_console_putchar(int ch)` 函数，第一个参数 `arg0` 需要传入待打印字符的ASCII码，第二、三个参数的值可直接设为0。测试时，调用 `sbi_call(1,0x30,0,0)` 将会输出字符'0'。其中1代表输出，0x30代表'0'的ascii值，arg1-2不需要输入，可以用0填充。

Type	Function	Function ID
Timer	sbi_set_timer	0
Console	sbi_console_putchar sbi_console_getchar	1 2
IPI	sbi_clear_ipi sbi_send_ipi	3 4
Memory Model	sbi_remote_fence_i sbi_remote_sfence_vma sbi_remote_sfence_vma_asid	5 6 7
Shutdown	sbi_shutdown	8

3.编写字符串打印函数（40%）

需要在`./arch/riscv/libs/print.c`文件中通过调用`sbi_call()`实现字符串打印函数`int puts(char* str)`及数字打印函数`int put_num(uint64_t n)`，后者可将数字转换为字符串后调用前者执行。

```
#./arch/riscv/libs/print.c
#include "defs.h"
extern sbi_call();
int puts(char* str){
    int i = 0 ;
    while( str[i] != '\0' ){
        sbi_call(1, str[i],0,0 );
        i += 1;
    }
    return 0;
}

int put_num(uint64_t n){
    //convert  unsigned long long uint64_t to string
    char * str ;
    int i = 0;
    int k = 0;
    char temp;//临时变量，交换两个值时用到
    char index[] = "0123456789";
    uint64_t unum = n ;
    unsigned int radix = 10 ;
    str[i++] = index[unum % radix];
    unum /= radix;
    while(unum > 0){
        str[i++] = index[unum % radix];
        unum = unum/  radix;
    }
    for(int j = k;j <= (i-1)/2; j++)//头尾一一对称交换，i其实就是字符串的长度，索引最大值比长度少1
    {
        temp=str[j];//头部赋值给临时变量
        str[j]=str[i-1+k-j];//尾部赋值给头部
        str[i-1+k-j]=temp;//将临时变量的值(其实就是之前的头部值)赋给尾部
    }
    str[i] = '\0';
    puts(str) ;
    return 0;
}
```

3.4 编译及测试

在`lab1/arch/riscv/kernel/main.c`中将`21922192`修改为你的学号，在项目最外层输入`make run`命令调用Makefile文件完成整个工程的编译及执行。

如果编译失败，及时使用`make clean`命令清理文件后再重新编译。

如果程序能够正常执行并打印出相应的字符串及你的学号，则实验成功。预期的实验结果如下，请附上相应的命令行及实验结果截图。

```
oslab@3c1da3906541:~/lab1$ make run
Hello RISC-V!
21922192
```

```
oslab@878ac559f77d:~/lab1$ make run
make -C arch/riscv all
make[1]: Entering directory '/home/oslab/lab1/arch/riscv'
make -C kernel
make[2]: Entering directory '/home/oslab/lab1/arch/riscv/kernel'
make[2]: Nothing to be done for 'all'.
make[2]: Leaving directory '/home/oslab/lab1/arch/riscv/kernel'
make -C libs
make[2]: Entering directory '/home/oslab/lab1/arch/riscv/libs'
make[2]: Nothing to be done for 'all'.
make[2]: Leaving directory '/home/oslab/lab1/arch/riscv/libs'
make[1]: Leaving directory '/home/oslab/lab1/arch/riscv'
riscv64-unknown-elf-ld -T arch/riscv/kernel/vmlinux.lds arch/riscv/
riscv64-unknown-elf-objcopy -O binary vmlinux arch/riscv/boot/Image
nm vmlinux > System.map

OpenSBI v0.6
   ____                    _____ ____ _____
  / __ \                  / ____|  _ \_   _|
 | |  | |_ __   ___ _ __ | (___ | |_) || |
 | |  | | '_ \ / _ \ '_ \ \___ \|  _ < | |
 | |__| | |_) |  __/ | | |____) | |_) || |_
  \____/| .__/ \___|_| |_|_____/|____/_____|
        | |
        |_|

Platform Name          : QEMU Virt Machine
Platform HART Features : RV64ACDFIMSU
Platform Max HARTs     : 8
Current Hart           : 0
Firmware Base          : 0x80000000
Firmware Size          : 120 KB
Runtime SBI Version    : 0.2

MIDELEG : 0x0000000000000222
MEDELEG : 0x000000000000b109
PMP0    : 0x0000000080000000-0x000000008001ffff (A)
PMP1    : 0x0000000000000000-0xffffffffffffffff (A,R,W,X)
Hello riscv
3180103721
```

4 讨论和心得

请在此处填写实验过程中遇到的问题及相应的解决方式。

由于本实验为新实验，可能存在不足之处，欢迎同学们对本实验提出建议。

问题1: `docker run -it -v C:\Users\12638\Desktop\operating system\lab\lab1:/home/oslab/lab1 -u oslab -w /home/oslab  fa20 /bin/bash` 一开始是这么输入的, 但是出错 docker: invalid reference format. 看了一下问题应该是C:后面就直接匹配了映射所以出错. 原来 `pwd` 可以直接自动填入目前的路径, 非常好用, 感谢实验让我学会了很多命令的输入方法.

问题2 : touch: cannot touch 'testlfile': Permission denied

ls -al 看了一下好像oslab用户没有写权限, 那就sudo chmod 777 lab1 但是还有密码.密码不是oslab

只好开个root 看看 `cat /etc/shadow`, 密码非常长, `passwd oslab` 修改成123.

```
root@db2d99121766:/home/oslab# passwd -S oslab
oslab P 09/26/2020 0 99999 7 -1
root@db2d99121766:/home/oslab# cat /etc/shadow
root:*:18355:0:99999:7:::
daemon:*:18355:0:99999:7:::
bin:*:18355:0:99999:7:::
sys:*:18355:0:99999:7:::
sync:*:18355:0:99999:7:::
games:*:18355:0:99999:7:::
man:*:18355:0:99999:7:::
lp:*:18355:0:99999:7:::
mail:*:18355:0:99999:7:::
news:*:18355:0:99999:7:::
uucp:*:18355:0:99999:7:::
proxy:*:18355:0:99999:7:::
www-data:*:18355:0:99999:7:::
backup:*:18355:0:99999:7:::
list:*:18355:0:99999:7:::
irc:*:18355:0:99999:7:::
gnats:*:18355:0:99999:7:::
nobody:*:18355:0:99999:7:::
_apt:*:18355:0:99999:7:::
messagebus:*:18526:0:99999:7:::
oslab:$6$wC6SoyyS$wKGGtRoKQN88dcBF1F9JDuB95z.7/pk8Ishl1vXFrjuSnIIIiigQPDzJMxPkgpvsMACDgw6X0QKfcsOGtFZBn.:18531:0:99999:7:::
```

但是, 好像再次启动run密码又不一样? 应该是停止容器后进入而不能再run

再密码改成o. run一次之后应该用 docker exec , 因为run 会重新开一个container.

问题3 : 但是没有映射上去. 创建了文件不会在本地出现.

docker: Error response from daemon: c\Users\12638\Desktop\operatingSystem\lab\lab1\%! (EXTRA string=is not a valid Windows path).

用pwd会出错，我用绝对路径来mount

问题4:联合汇编, 输出操作数和输入操作数看不懂了. 不知道怎么用的. 于是阅读例子

```c
int main(void) {
    int sum;
    int add1 = 100 ;
    int add2 = 200;
    __asm__ volatile (
        #汇编指令
        "add %[dest],%[src1],%[src2]"
        #输出操作数
        : [dest] "=r" (sum) // 把C程序变量和add的操作数绑定
        #输入操作数
        : [src1] "r" (add1), [src2] "r" (add2)
        #可能影响的寄存器或存储器
        : "memory"
    );
    return ret_val;
}
```

怎么使用ecall指令发起sbi调用请求?

手册没说清楚, 就说通过引发环境调用异常来请求执行环境.

问题5 :print.c中用了 sprintf, 那么makefile中怎么引入它的头文件呢?makefile的 all又应该怎么写呢? 按道理应该和kernel的makefile差不多.为什么要把 每个.S .c 替换为 .o?

CFLAG是什么? cflag就是可以加一堆参数在前面, 在父makefile中定义了.

问题6:error: 'type' undeclared (first use in this function)

```
oslab@878ac559f77d:~/lab1$ make
make -C arch/riscv all
make[1]: Entering directory '/home/oslab/lab1/arch/riscv'
make -C kernel
make[2]: Entering directory '/home/oslab/lab1/arch/riscv/kernel'
riscv64-unknown-elf-gcc  -g -march=rv64imafd -mabi=lp64 -mcmodel=medany -ffunct
ostdlib -nostdinc -fno-builtin -static -lgcc  -I/home/oslab/lab1/include -I/hom
 -c sbi.c
sbi.c: In function 'sbi_call':
sbi.c:13:23: error: 'type' undeclared (first use in this function)
   13 |         : [type] "r" (type), [src0] "r" (arg0), [src1] "r" (arg1), [src
      |                       ^~~~
sbi.c:13:23: note: each undeclared identifier is reported only once for each fu
Makefile:12: recipe for target 'sbi.o' failed
make[2]: *** [sbi.o] Error 1
make[2]: Leaving directory '/home/oslab/lab1/arch/riscv/kernel'
Makefile:2: recipe for target 'all' failed
make[1]: *** [all] Error 2
make[1]: Leaving directory '/home/oslab/lab1/arch/riscv'
Makefile:32: recipe for target 'vmlinux' failed
make: *** [vmlinux] Error 2
oslab@878ac559f77d:~/lab1$
```

改成*sbi_type* ,然后make clean

在clean中也可以clean: cat print.c

问题7 : make run 后一直打印乱码

sbi_call(1, *str*,0,0); 这里不能直接传入指针, 要一个个读入char 然后输出.

可以用上节课的gdb试一试单步调试.

```
0x0000000000001000 in ?? ()
(gdb) b *0x80200000
Breakpoint 1 at 0x80200000: file head.S, line 8.
(gdb) info b
Num     Type           Disp Enb Address            What
1       breakpoint     keep y   0x0000000080200000 head.S:8
(gdb) continue
Continuing.

Breakpoint 1, _start () at head.S:8
8               la sp,stack_top
(gdb) stepi
0x0000000080200004 in _start () at head.S:8
8               la sp,stack_top
```

问题8: 不断打印hello riscv

```
put_num (n=3180103721) at print.c:17
17              int i = 0;
(gdb)
18              char index[]="0123456789";
(gdb)
19              uint64_t unum = n ;
(gdb)
20              int radix = 10 ;
(gdb) p unum
$3 = 3180103721
(gdb) s
22                      str[i++] = index[unum % radix];
(gdb)

Breakpoint 1, _start () at head.S:8
8               la sp,stack_top
(gdb)
_start () at head.S:9
9               call main
(gdb)
main () at main.c:7
7           puts("Hello riscv \n");
(gdb)
```

gdb看, put_num打印执行了一部分又跳转到上面的puts.

为什么`str[i++] = index[unum % radix];`之后不执行`unum /= radix;`而是跳转到 Breakpoint 1, _start () at head.S:8
8 la sp,stack_top

我把内联汇编最后的"memory"去掉就好了